

Operations Research

Column generation & approximation algorithms

Prof. Dr. Martin Bichler

Department of Computer Science

School of Computation, Information, and Technology

Technical University of Munich

Course outline

- **Linear Optimization**

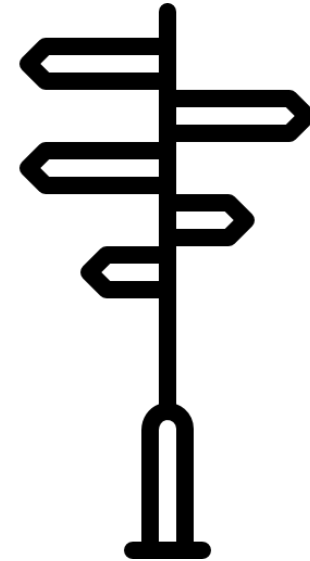
- Modeling by linear programming and graphical solution
- Solving linear programs, convexity
- Simplex algorithm, 2-phase simplex
- Simplex algorithm in matrix notation
- Sensitivity analysis, reoptimization
- Duality theory, zero-sum games

- **Integer Linear Optimization**

- Modeling integer programming problems (IPs)
- Solving IPs: branch & bound, cutting planes
- **Advanced solution methods: column generation, approximation algorithms**
- Fast solvable integer problems: total unimodularity, matroids
- Network flow problems
- Traveling salesperson problem

- **Nonlinear Optimization**

- Optimization of multidimensional functions
- Convex optimization



Today you will learn...

- how to use **column generation** to optimally solve large integer optimization problems.
- **approximation algorithms** for *NP*-hard optimization problems.
- the **Shapley-Folkman-Starr Lemma**.



Cutting stock problem

A DIY store offers in its assortment of wooden boards the following lengths: 3m, 5m and 9m.

Customers want 25 3m boards, 20 5m boards and 15 9m boards.

For production, 17m boards are available from the producer. The aim is to generate as little waste as possible. How should the 17m boards be cut?

Possible model

Parameter:

b_i : number of required boards of length l_i

Variables:

x_j : from the big board $j \in K$ with length L smaller boards are cut off

y_{ij} : number of produced boards of length l_i that are cut off from board j

$$\min \sum_{j \in K} x_j$$

$$3y_{1j} + 5y_{2j} + 9y_{3j} \leq 17x_j \quad \forall j \in K$$

$$\sum_{j \in K} y_{1j} \geq 25$$

$$\sum_{j \in K} y_{2j} \geq 20$$

$$\sum_{j \in K} y_{3j} \geq 15$$

$$x_j \in \{0,1\} \quad \forall j$$

$$y_{ij} \in \mathbb{N}_0 \quad \forall i,j$$

Possible model: a bin packing formulation

Parameter:

b_i : number of required boards of length l_i

Variables:

x_j : from the big board $j \in K$ with length L smaller boards are cut off

y_{ij} : number of produced boards of length l_i that are cut off from board j

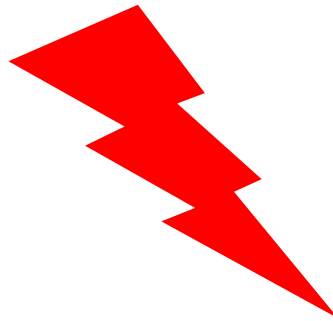
$$\min \sum_{j \in K} x_j$$

$$\sum_i l_i y_{ij} \leq L x_j \quad \forall j \in K$$

$$\sum_{j \in K} y_{ij} \geq b_i \quad \forall i$$

$$x_j \in \{0,1\} \quad \forall j$$

$$y_{ij} \in \mathbb{N}_0 \quad \forall i, j$$



Relaxation yields poor bounds

→ Difficulties for solvers based on branch-and-bound

Alternative model

Enumerate all possible patterns that are possible with a board of length L .

For our example with
 $L = 17$
 and boards of lengths
 3, 5, 9

	3m boards	5m boards	9m boards
Pattern 1	5	0	0
Pattern 2	4	1	0
Pattern 3	2	2	0
Pattern 4	2	0	1
Pattern 5	1	1	1
Pattern 6	0	3	0
Pattern 7	0	0	1
⋮	⋮	⋮	⋮

Example

We consider the example from before with the following patterns:

	3m-boards	5m-boards	9m-boards
Pattern 1	5	0	0
Pattern 2	4	1	0
Pattern 3	2	2	0
Pattern 4	2	0	1
Pattern 5	1	1	1
Pattern 6	0	3	0
Pattern 7	0	0	1

$$\begin{aligned}
 \min \quad & x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 \\
 \text{s.t.} \quad & 5x_1 + 4x_2 + 2x_3 + 2x_4 + x_5 \geq 25 \quad (3 \text{ m constraint}) \\
 & x_2 + 2x_3 + x_5 + 3x_6 \geq 20 \quad (5 \text{ m constraint}) \\
 & x_4 + x_5 + x_7 \geq 15 \quad (9 \text{ m constraint}) \\
 & x_j \geq 0 \quad \text{for all } j
 \end{aligned}$$

Modeling with patterns

Parameter:

a_{ij} : number of boards of length l_i in pattern j

Variables:

x_j : number of times pattern j is produced
(let Γ be the set of possible patterns)

$$\min \sum_{j \in \Gamma} x_j$$

$$\sum_{j \in \Gamma} a_{ij} x_j \geq b_i \quad \forall i$$

$$x_j \in \mathbb{N}_0 \quad \forall j$$

Advantage:

strong LP relaxation

Disadvantage:

many patterns (and thus variables)
are possible

Problem

Cutting stock problems are often very large in practice:

- many cutting patterns $\Rightarrow$ many columns ($= n$)
- we might not be able to keep all columns in the main memory
- we might not want to compute all objective function coefficients

$$c'_N = -(c_N - N'^T c_B) \quad (N' = B^{-1}N)$$

But:

Often, there are a few constraints compared to the number of variables

$\rightarrow$ only $m \ll n$ basic variables

$\rightarrow$ only a few patterns are really needed

Can we find the optimal basis without explicitly enumerating all the patterns?

To do so, let us first consider the LP relaxation of our problem!

How many variables do we really need?

Can the columns for the new base variable be generated?

- Suppose an initial basis $\{x_1, x_6, x_7\}$ in the last example is known, but not the other patterns.

→ We do not consider the entire matrix N , but look for only one column N_j (i.e. a variable x_{N_j}) that improves the objective function value when it is included (**prices**) in the basis.

Simplex tableau

Observation:

In simplex, directly applied to **minimization problems**, the goal is to make all objective function coefficients **non-positive**. That is, the column with the largest c_i is usually chosen as the pivot column. Everything else remains the same.

Basic variable	Coefficient:							Solution
	Z	x_{B_1}	...	x_{B_m}	x_{N_1}	...	$x_{N_{n-m}}$	
Z	1	0		0	$\mathbf{c}'_N = -(\mathbf{c}_N - \mathbf{N}'^T \mathbf{c}_B)^T$			$\mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{b}$
x_{B_1}		1			$\mathbf{N}' = \mathbf{B}^{-1} \mathbf{N}$			$\mathbf{b}' = \mathbf{B}^{-1} \mathbf{b}$
...			...					
x_{B_m}				1				

Revised simplex

$$\begin{aligned}
 \min \quad & x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 \\
 \text{s.t.} \quad & 5x_1 + 4x_2 + 2x_3 + 2x_4 + x_5 \geq 25 \text{ (3m constraint)} \\
 & x_2 + 2x_3 + x_5 + 3x_6 \geq 20 \text{ (5m constraint)} \\
 & x_4 + x_5 + x_7 \geq 15 \text{ (9m constraint)} \\
 & x_j \geq 0 \text{ for all } j
 \end{aligned}$$

First basis = (x_1, x_6, x_7)

$$B = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad B^{-1} = \begin{pmatrix} \frac{1}{5} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$c'_N = -c_N + (B^{-1}N)^T c_B \quad c_B^T B^{-1} = (1 \ 1 \ 1) \begin{pmatrix} \frac{1}{5} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & 1 \end{pmatrix} = \left(\frac{1}{5} \ \frac{1}{3} \ 1 \right)$$

Choose the entering variable (column)

Each variable corresponds to a cutting pattern.

Choose a pattern that maximizes the value of the non-basic variable.

$$c_B^T B^{-1} N_j - c_{N_j} \Rightarrow c_B^T B^{-1} \begin{bmatrix} a_{3m} \\ a_{5m} \\ a_{9m} \end{bmatrix} - 1 = \begin{bmatrix} \frac{1}{5} & \frac{1}{3} & 1 \end{bmatrix} \cdot \begin{bmatrix} a_{3m} \\ a_{5m} \\ a_{9m} \end{bmatrix} - 1$$

We are familiar
with this from
sensitivity
analysis

We choose a pattern such that $\frac{1}{5}a_{3m} + \frac{1}{3}a_{5m} + a_{9m} - 1$ is positive.

Pricing Problem

We choose a pattern such that $\frac{1}{5}a_{3m} + \frac{1}{3}a_{5m} + a_{9m} - 1$ is positive.

→ Solve

$$\begin{aligned} \max \quad & \frac{1}{5}a_{3m} + \frac{1}{3}a_{5m} + a_{9m} - 1 \\ \text{s. t.} \quad & 3a_{3m} + 5a_{5m} + 9a_{9m} \leq 17 \\ & a_{3m}, a_{5m}, a_{9m} \in \mathbb{N}_0 \end{aligned}$$

→ If the optimal solution of this **pricing problem** is positive, then the corresponding pattern improves the solution of the original problem.

$$z^* = \frac{8}{15}, a_{3m} = a_{5m} = a_{9m} = 1 \Rightarrow \text{pattern 5, i.e. } x_5 \text{ is the new basic variable!}$$

- x_5 does not even need to be explicitly listed in the model (and in the LP formulation) beforehand.
- We can simply generate the new variable if it turns out to improve the objective function value!

Revised Simplex

Exiting non-basic variable

$$\mathbf{b}' = \mathbf{B}^{-1}\mathbf{b}$$

$$\mathbf{b}' = \mathbf{B}^{-1}\mathbf{b} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{5} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 25 \\ 20 \\ 15 \end{pmatrix} = \begin{pmatrix} 5 \\ 20 \\ \frac{3}{15} \end{pmatrix}$$

$$\mathbf{N}' = \mathbf{B}^{-1}\mathbf{N}$$

$$\mathbf{B}^{-1} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{5} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{1}{5} \\ \frac{1}{3} \\ 1 \end{pmatrix}$$

$$\text{Basis} \begin{pmatrix} x_1 \\ x_6 \\ x_7 \end{pmatrix} \rightarrow \begin{pmatrix} 5 \\ 20 \\ \frac{3}{15} \end{pmatrix} / \begin{pmatrix} \frac{1}{5} \\ \frac{1}{3} \\ 1 \end{pmatrix} = \begin{pmatrix} 25 \\ 20 \\ 15 \end{pmatrix} \quad x_7 \text{ as exiting basic variable}$$

Old basis = (x_1, x_6, x_7)

New basis = (x_1, x_6, x_5)

$$\text{New basis} = \begin{pmatrix} 5 & 0 & 1 \\ 0 & 3 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

Another iteration

$$c_B^T B^{-1} = [1 \quad 1 \quad 1] \cdot \begin{bmatrix} \frac{1}{5} & 0 & -\frac{1}{5} \\ 0 & \frac{1}{3} & -\frac{1}{3} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{5} & \frac{1}{3} & \frac{7}{15} \end{bmatrix} \quad \text{New basis} = \begin{pmatrix} 5 & 0 & 1 \\ 0 & 3 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

$$c_B^T B^{-1} N_j - c_N \Rightarrow c_B^T B^{-1} \begin{bmatrix} a_{3m} \\ a_{5m} \\ a_{9m} \end{bmatrix} - 1 = \begin{bmatrix} \frac{1}{5} & \frac{1}{3} & \frac{7}{15} \end{bmatrix} \cdot \begin{bmatrix} a_{3m} \\ a_{5m} \\ a_{9m} \end{bmatrix} - 1$$

$$\begin{aligned} \max \quad & \frac{1}{5} a_{3m} + \frac{1}{3} a_{5m} + \frac{7}{15} a_{9m} - 1 \\ \text{s.t.} \quad & 3a_{3m} + 5a_{5m} + 9a_{9m} \leq 17 \\ & a_{3m}, a_{5m}, a_{9m} \in \mathbb{N}_0 \end{aligned}$$

$z^* = \frac{2}{15}$, $a_{3m} = 4$, $a_{5m} = 1$, $a_{9m} = 0 \rightarrow$ pattern 2, i.e. x_2 is the new basic variable!

Column generation by the dual problem

Choose a subset of all variables such that the associated constrained LP is feasible.

Do {

- Solve the dual problem to get the associated shadow price vector $y^T = c_B^T B^{-1}$.
- Solve the pricing problem.
- If there is an improving column, add it to the LP.
(This creates a new constraint in the dual).
- Otherwise, stop.

}

The optimal solution of this LP is then also optimal for the LP with all variables.

Column generation

To decide whether adding a variable improves the objective value, we formulate the **pricing problem** whose solution characterizes the variable sought.

The pricing problem should be quickly solvable.

In the cutting stock problem, this is the knapsack problem.

- Although the knapsack problem is NP-hard, it can often be solved fast.

The pricing problem must also provide information about whether the current solution is optimal. (In the example: If the maximum value is less than 0, there is no improving variable)

Column generation in IP (Branch & Price)

Column generation can be applied only for the continuous relaxation (we need the tools from duality to generate the columns).

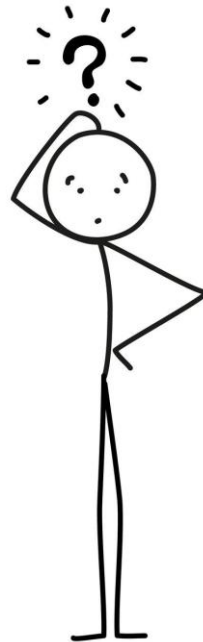
If the solution is not integral, we can use branch-and-bound as in modeling with few variables, and branch over the number of each pattern.

Branch-and-Bound with column generation in the nodes: **Branch-and-Price**.

Other examples for column generation

- Scheduling
 - Variables are possible work plans
- Tour planning
 - Variables are all possible tours
- Set cover
- Kidney exchange
-

Explain why it is possible to solve an optimization problem optimally without formulating it completely.



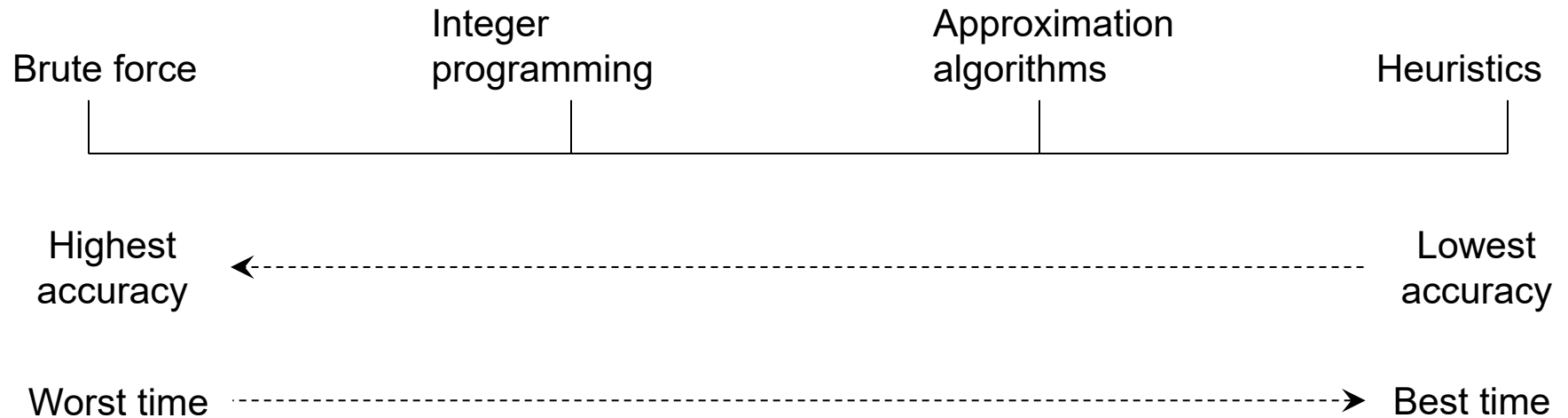
Today you will learn...

- how to use **column generation** to optimally solve large integer optimization problems.
- **approximation algorithms** for *NP*-hard optimization problems.
- the **Shapley-Folkman-Starr Lemma**.



Solving NP-hard optimization problems

- Integer programming
- Approximation algorithms
- Heuristics



Heuristics

- based on intuition or common sense
- usually very time efficient
- no solid mathematical basis
- do not guarantee an optimal solution
- sometimes result in "quite" good solutions

Heuristic for the knapsack problem

$$\begin{array}{ll}
 \max & \sum_{j=1}^n v_j x_j \\
 \text{s.t.} & \sum_{j=1}^n w_j x_j \leq W \\
 & x_j \in \{0, 1\} \quad \text{for } j = 1, \dots, n
 \end{array}$$

v_j : value of item i
 w_j : weight of item i
 W : maximal weight of the knapsack

Idea of a heuristic:

- Sort the items in descending order by their value-per-weight $\frac{v_j}{w_j}$.
- Iteratively add the item with the current highest value-per-weight to the backpack as long as it fits.
- For the LP, this would be optimal (with the fractional portion of the first item no longer fitting)
- When items are comparatively small to the knapsack volume, the heuristic gives good results.
- How bad can the result of the heuristic be?

Metaheuristics

Metaheuristics are general algorithms which are applied to a variety of problems in combinatorial optimization to find good solutions to problems. Heuristics, in contrast, are problem-specific.

Example of metaheuristics:

- Local search
- Simulated annealing
- Tabu search
- Evolutionary algorithms
- Ant colony optimization

Local search for the knapsack problem

Start with any feasible solution to the knapsack problem.

While (termination criterion not yet fulfilled){

- Add a new item to the knapsack.
 - As long as the knapsack is too full, throw items that are not newly added out of the knapsack (e.g., by value-per-weight)
- }

→ no guarantee for the quality of the solution

→ can get into local optima (less problematic with simulated annealing and other randomized variants)

Approximation algorithms

Approximation algorithms solve *NP*-hard optimization problems.

- Good approximation algorithms are known for independent set, vertex cover, knapsack problem, max-SAT, etc.

The algorithms run in polynomial time.

Provide (worst-case) bounds for the quality of the solution.

- These are called approximation factors or ratios.

Approximation ratios

- For an instance I of an optimization problem, let
 - $OPT(I)$ be the objective function value of the optimal solution of I .
 - $ALG(I)$ be the objective function value that is computed by the approximation algorithm.
- An algorithm for a **minimization problem** has an **approximation ratio** $c > 1$ if it computes for all problem instances a value of at most $c \cdot OPT(I)$.
- An algorithm for a **maximization problem** has an **approximation ratio** c if it computes for all problem instances a value of at least $OPT(I)/c$.
- The class of problems with constant approximation ratio is called **APX** (a subset of **NP**).

Types of approximation algorithms

1. APX: class of problems that have a constant factor approximation.

- Approximation ratio: $\frac{ALG}{OPT} \leq c$ for minimization.
- Approximation ratio: $\frac{OPT}{ALG} \leq c$ for maximization. **Example: min vertex cover**

2. PTAS: Polynomial-Time Approximation Scheme.

- Approximation ratio $1 + \varepsilon$ for each $\varepsilon > 0$, where ALG runs in polynomial time in n . The runtime can be exponential in $1/\varepsilon$, e.g., $O(n^{f(\frac{1}{\varepsilon})})$.

3. FPTAS: Fully Polynomial-Time Approximation Scheme.

- Approximation ratio $1 + \varepsilon$ for each $\varepsilon > 0$, where ALG runs in polynomial time in n and $1/\varepsilon$, e.g., $O\left(\left(\frac{1}{\varepsilon}\right)^m n^k\right)$, where k and m are constants.

Example: DP for knapsack problem

Example of an approximation algorithm for the knapsack problem

Create two possible knapsack packs S_1 and S_2 .

S_1 is the result of our previous heuristics:

- Sort the items in descending order by their value-per-weight $\frac{v_j}{w_j}$.
- Iteratively add the item with the current highest value-per-weight to the knapsack as long as it fits.

S_2 includes only the highest value item that is not in the knapsack.

Select the backpack with the higher value as the solution.

Theorem:

The modified greedy algorithm for the knapsack problem has an approximation factor of 2.

Proof of the approximation ratio

Let the objects be sorted by value-per-weight, where $\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \dots$.

Let $1, \dots, k-1$ be the items that are included in S_1 .

$\sum_{i=1, \dots, k-1} v_i$ is the total value of S_1 , v_k is the total value of S_2 .

At least one of the terms $\sum_{i=1, \dots, k-1} v_i$ or v_k is at least $\frac{1}{2} OPT$.

The optimal fractional solution includes $1, \dots, k-1$ and a fraction α of item k .

$$\sum_{i=1, \dots, k-1} v_i + \alpha v_k = OPT_{fractional} \text{ with } \alpha < 1$$

Since the fractional solution has a total value at least as high as the total value of the integral solution, it holds:

$$OPT \leq OPT_{fractional} \leq \sum_{i=1, \dots, k-1} v_i + v_k \leq 2 \max \left\{ \sum_{i=1, \dots, k-1} v_i, v_k \right\} = 2ALG$$

Example with capacity 100

Item	Weight	Value	Value/weight
1	49	99	2.0204
2	2	4	2
3	51	101	1.9804

ALG selects {1,2} with value 103

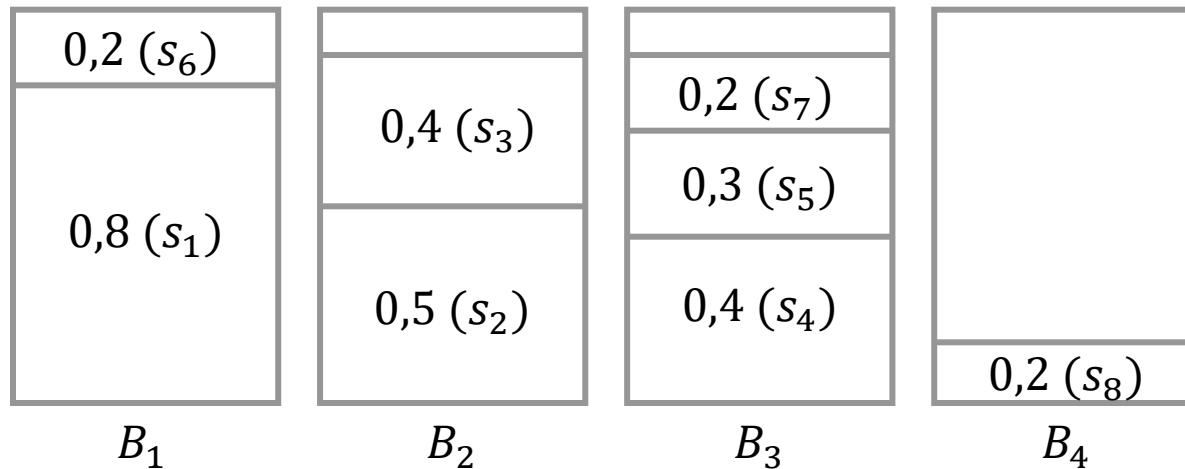
OPT is {1,3} with value 200

Example of an approximation algorithm for bin packing: First Fit Decreasing

Places an object in the first bin it fits. For large problems, at most 22% worse than the optimal solution in time $O(n \log n)$.

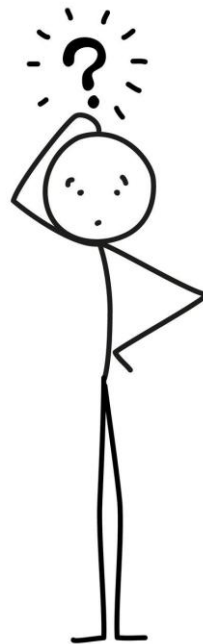
Items are sorted in descending order first.

$$S = \{0,8; 0,5; 0,4; 0,4; 0,3; 0,2; 0,2; 0,2\}$$



*Martello & Toth: Knapsack Problems, 1990, S. 224.
 $FFD \leq 11/9 OPT + 4$

What distinguishes heuristics from meta-heuristics and approximation algorithms?



Vertex cover

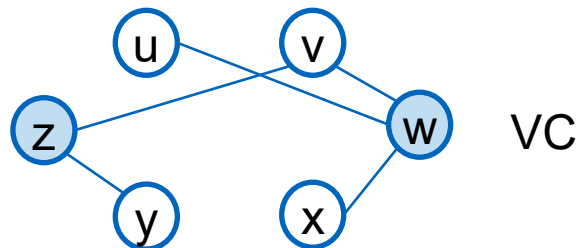
The vertex cover of a graph $G = (V, E)$ is a subset $W \subseteq V$ **containing at least one end node of each edge**, i.e. for each edge $(u, v) \in E$, $u \in W$ or $v \in W$.

OPT-VERTEX-COVER (VC):

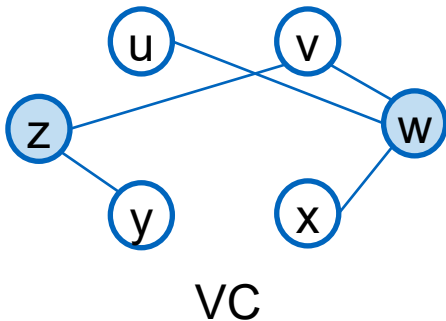
Given a graph G . Find the vertex cover of G with the smallest cardinality.

Theorem: The decision version of VC is **NP**-complete.

Application example: Assigning a constrained number of resources (e.g., ATMs, fire stations, or Wi-Fi routers) to junctions in a way that minimizes deployment costs while guaranteeing that no street, zone, or block is left unserved.



A 2-approximation for vertex cover



Algorithm VertexCoverApprox(G)

Input: G

Output: vertex cover C for G

$C \leftarrow \emptyset$

while G still has edges **do**

 choose an edge $e = (v, w)$ of G

 include nodes v and w to C

for each edge f incident to v or w **do**
 remove f from G

return C

Each chosen edge e has two end nodes in the set C .

But e must be covered in an optimal cover.

So one end node of e must be in OPT .

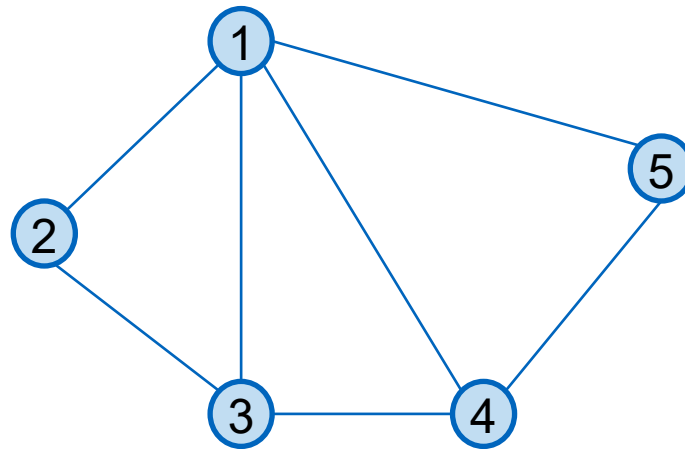
Therefore, there are at most 2x as many nodes in C as in OPT .

C is a 2-approximation of OPT .

Run time: $O(|E|)$

No better approximation algorithm is known so far.

Example



VC is APX-complete

Vertex cover can be approximated with a constant factor.

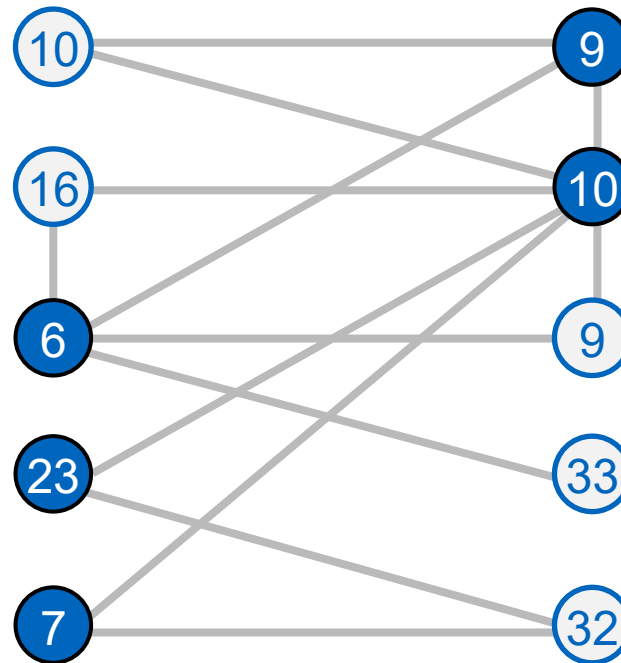
Vertex cover is **APX-complete**, i.e. the optimal solution cannot be approximated arbitrarily well (i.e. no PTAS exists), except if $P = NP$.

A problem is APX-hard if there is a PTAS reduction from any problem in APX to that problem. Proof for vertex cover is from 2005.

In bipartite graphs, the VC problem becomes the bipartite maximum matching (the assignment problem), which can be solved in polynomial time.

Weighted vertex cover

Given a graph $G = (V, E)$ with vertex weights $w_i \geq 0$, find a subset $C \subseteq V$ with the **smallest** total weight such that each edge is adjacent to at least one node in C .



$$\text{Weight} = 6 + 23 + 7 + 9 + 10 = 55$$

Weighted vertex cover as IP

Problem: Given a graph $G = (V, E)$ with vertex weights $w_i \geq 0$, find a subset $C \subseteq V$ with the **smallest** total weight such that each edge is adjacent to at least one node in C .

Integer program:

Each vertex i is represented by a 0/1 variable x_i :

$x_i = 1$ if vertex i is element of the vertex cover, else 0.

$C = \{i \in V \mid x_i = 1\}$.

Objective function: $\min \sum_i w_i x_i$.

For each edge i or j (or both) must be in C : $x_i + x_j \geq 1$.

Weighted vertex cover: approximation via LP

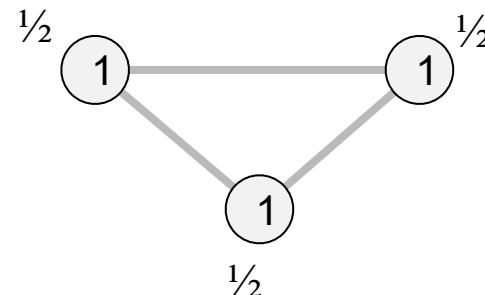
$$\begin{aligned}
 (IP) \quad & \min \sum_{i \in V} w_i x_i \\
 \text{s.t.} \quad & x_i + x_j \geq 1 \quad \forall (i, j) \in E \\
 & x_i \in \{0, 1\} \quad \forall i \in V
 \end{aligned}$$

If x^* is the optimal solution of (IP), then $C = \{i \in V \mid x_i^* = 1\}$ is a vertex cover with minimum weight.

The LP relaxation (LPR) can be computed in polynomial time.
Can it help us to find a good approximation?

$$\begin{aligned}
 (LPR) \quad & \min \sum_{i \in V} w_i x_i \\
 \text{s.t.} \quad & x_i + x_j \geq 1 \quad \forall (i, j) \in E \\
 & 1 \geq x_i \geq 0 \quad \forall i \in V
 \end{aligned}$$

We can **round** the fractional solution!



Weighted vertex cover: LP-based rounding

Lemma: If x^{LPR} is the optimal solution for (LPR), then $C = \{i \in V \mid x_i^* \geq 1/2\}$ is a vertex cover whose weight is at most 2x the minimum possible weight.

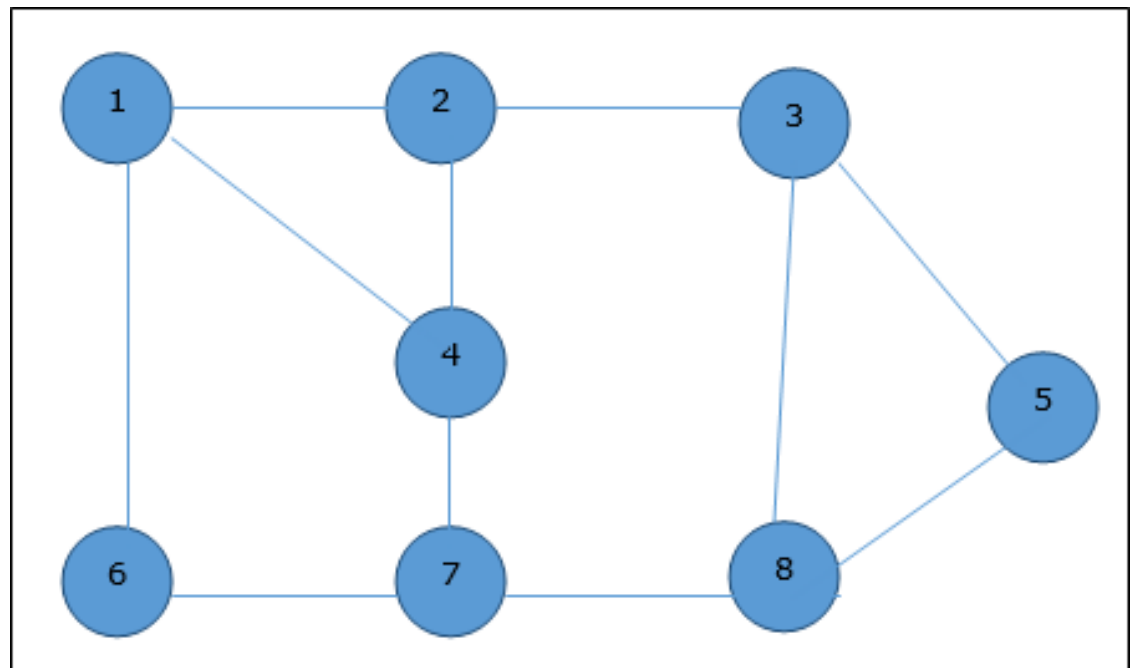
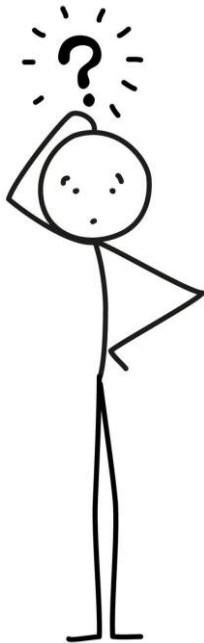
Proof: Consider an edge $(i, j) \in E$.

- Since $x_i^{LPR} + x_j^{LPR} \geq 1$, either $x_i^{LPR} \geq 1/2$ or $x_j^{LPR} \geq 1/2 \Rightarrow (i, j)$ covered.
- This results in a valid vertex cover since a vertex is contained in C for every (i, j) .
- Let x^{IP} be an optimal vertex cover, and x^R the rounded solution.
- Claim: $w(x^R) \leq 2w(x^{LPR})$
- Proof: $w(x^R) = \sum_{i \in V} w_i x_i^R = \sum_{\{i \in V \mid x_i^{LPR} \geq 1/2\}} w_i \leq \sum_{i \in V} w_i 2x_i^{LPR} = 2w(x^{LPR})$
 - The weights of the rounded vertices ≤ 2 x the LPR solution
- No integral solution can be better than x^{LPR} .

Theorem: The LP-based rounding algorithm is a 2-approximation.

Proof: Lemma + LP solvable in polynomial time.

Run the $\text{VertexCoverApprox}(G)$ program on the graph below. The numbers index the vertices and are not weights.



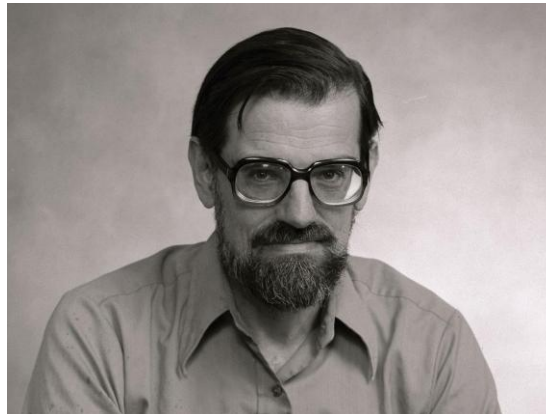
Today you will learn...

- how to use **column generation** to optimally solve large integer optimization problems.
- **approximation algorithms** for *NP*-hard optimization problems.
- the **Shapley-Folkman-Starr Lemma**.



Shapley-Folkman-Starr lemma

The Shapley-Folkman-Starr lemma provides a foundational theoretical result that supports the use of LP relaxations in solving ILPs. It explains why solutions to high-dimensional LP relaxations are often close to integer solutions



Lloyd Stowell Shapley
(1923 – 2016)
2012 Nobel Memorial Prize in
Economic Sciences



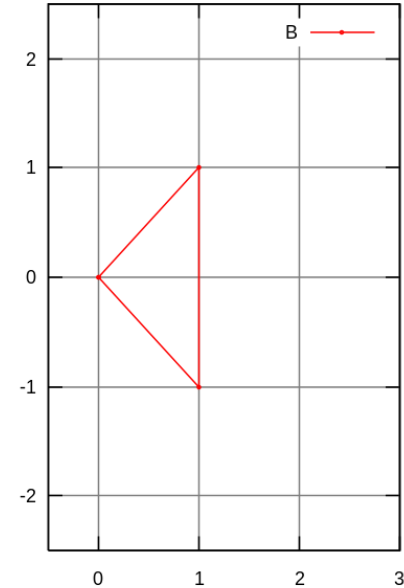
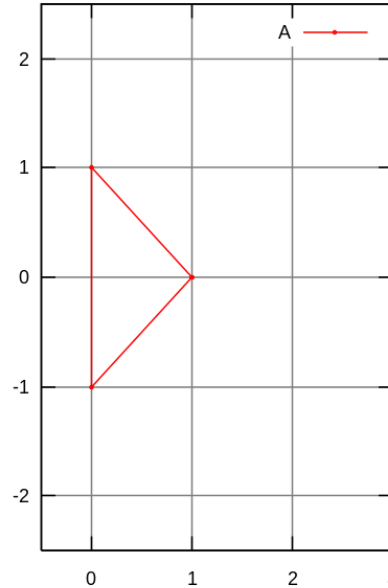
Hermann
Minkowski
1864-1909

Minkowski Sum

$$A + B = \{a + b \mid a \in A, b \in B\}$$

Set of A: $\{(1, 0), (0, 1), (0, -1)\}$

Set of B: $\{(0, 0), (1, 1), (1, -1)\}$



Minkowski Sum of Sets $(A + B)$:

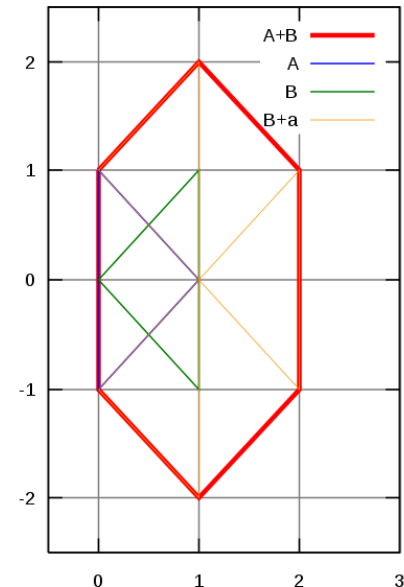
$\{(1, 0), (2, 1), (2, -1), (0, 1), (1, 2), (1, 0), (0, -1), (1, 0), (1, -2)\}$

For all subsets $A, B \subseteq X$ of a real vector space, X , the convex hull of their Minkowski sum is the Minkowski sum of their convex hulls:

$$\text{Conv}(A + B) = \text{Conv}(A) + \text{Conv}(B)$$

By induction:

$$\text{Conv}(\sum_{i=1}^n Q_i) = \sum_{i=1}^n \text{Conv}(Q_i)$$



Shapley-Folkman-Starr (Starr (1969))

$\{S^j\}_{j=1}^n$ is a collection of sets in $\mathbb{R}^m$ with $n > m$. These sets do not have to be convex.
 n is the number of sets, m is the dimension.

Minkowski sum of the convex hull: $S = \sum_{j=1}^n \text{conv}(S^j)$.

Lemma (Shapley-Folkman-Starr):

Every $b \in S$ can be expressed as $\sum_{j=1}^n x^j$ where $x^j \in \text{conv}(S^j)$ for all $j = 1, \dots, n$
 and $|\{j: x^j \in S^j\}| \geq n - m$.

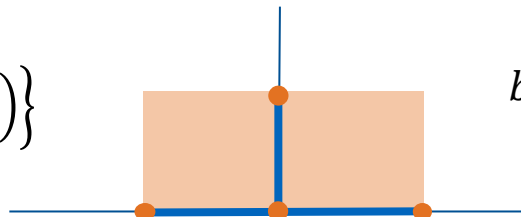
This means, a large part of the convex decomposition is in S^j , not just the convex hull S .

Example: $S^1, S^2, S^3 =$

$$\left\{ \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right\}, \left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right\}, \left\{ \begin{pmatrix} -1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right\}$$

Convex hull of S^j : 

Minkowski sum: 



$$b = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$n = 3, m = 2$

$$|\{j: x^j \in S^j\}| \geq 3 - 2 = 1$$

$\begin{pmatrix} 0 \\ 0 \end{pmatrix} \in S^1$ is a vector in S^1

Shapley-Folkman-Starr and 0-1 integer LPs

Lemma (Shapley-Folkman-Starr):

Every $b \in \text{conv}(S)$ can be expressed as $\sum_{j=1}^n x^j$ where $x^j \in \text{conv}(S^j)$ for all $j = 1, \dots, n$ and $|\{j: x^j \in S^j\}| \geq n - m$.

Let A be a $m \times n$ 0-1 matrix and $b \in \mathbb{R}^m$ with $n > m$. Denote each column j of the matrix A by a^j . $S^j = \{a^j, 0\}$.

Consider the problem

$$\max\{c^T x : Ax = b, x_j \in \{0,1\} \forall j = 1, \dots, n\}.$$

Let x^* be a solution to the linear relaxation of this problem.

This means b is in the Minkovsky sum of the convex hull of the column vectors S^j .

Then, the Shapley-Folkman-Starr lemma yields the existence of a vector $y \in [0, 1]$ such that at least $n - m$ components are $y \in \{0,1\}$.

This means, y has at most m fractional components.

Many columns (n) and a few constraints: there is a solution with only a few fractional variables.

$$A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \quad b^T = (1.5 \ 1.0) \quad y^T = (1 \ 0.5 \ 0.5)$$

Proof of the Shapley-Folkman-Starr Lemma

Shapley-Folkman-Starr: $b = \sum_{j=1}^n a^j y_j$ where each $y_j \in [0,1]$ with at least $n - m$ of them being either 0 or 1. Equivalently, y has at most m fractional components.

Proof: Suppose $b = \sum_{j=1}^n a^j y_j$ with at least $m + 1$ components of y being fractional (against the lemma).

Let C be a submatrix of A that corresponds to integer components of y .

Let C' be a submatrix of A that corresponds to fractional components of y .

$$b = C'y_{c'} + Cy_c \equiv b - Cy_c = C'y_{c'}$$

Suppose for a contradiction that C' has more columns ($\geq m + 1$) than rows (m), then the kernel (or null space) is nonempty.

$$\Rightarrow \ker(C') \neq 0 \Rightarrow \exists z \in \ker(C')$$

Note for $Az = 0$ there are infinitely many solutions.

Note that if $Az = 0$ then $A(x + z) = b$ and $A\lambda z = 0$

Consider $y_{c'} + \lambda z$: Now $b - Cy_c = C'[y_{c'} + \lambda z]$

Choose λ to make at least one component of $y_{c'}$ take 0 or 1.

After scaling λ , we call $y_{c'} + \lambda z = r$ and $b - Cy_c = C'r$

The number of fractional components is reduced by 1 and we still have a representation of b .

If $m \leq 1$, the only solution to $\ker(C')$ is $z = 0$.

So, there cannot be more than m fractional variables.

$$A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

$$b = (1.5 \ 1.0)$$

$$y = (1 \ 0.5 \ 0.5)$$

$$b - \begin{pmatrix} 1 \\ 0 \end{pmatrix} 1 = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}$$

$$b - Cy_c = C'y_{c'}$$

$$Az = 0 \Rightarrow \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$C'[y_{c'} + \lambda z] = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \left(\begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} + \lambda \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} \right)$$